BED 2 Cloud Services - Kata 1

Intro

Refresher after summer break. Try to avoid using AI for this one - use the references instead:

- Express: https://expressjs.com/en/5x/api/
- Jest: https://jestjs.io/docs/getting-started
- Supertest: https://www.npmjs.com/package/supertest
- dotenv: https://www.npmjs.com/package/dotenv

We're building this app the way it needs to be built for the cloud. Two things matter there: **external configuration** and **automated tests**. Config gets loaded from the environment instead of hardcoded, so the same code runs in different environments without changes. Tests matter because later we'll wire this into a CI/CD pipeline, and a pipeline decides pass/fail based on your tests, not on someone eyeballing the output. Keep both in mind as you go.

Stage 1: Basic Express API setup

Goal: a running Express server with a `/health` endpoint.

1. `npm i express`
2. `touch app.js` and build your Express app:
    - Hardcode `const PORT = 3000` for now.
    - Add an **async** `/health` route that responds with:

```
{
  status: 'ok',
  uptime: process.uptime(),
  timestamp: new Date().toISOString()
}
```

    - Call `app.listen(PORT, ...)` at the bottom of the file.
3. In `package.json`, add a `start` script pointing at `app.js`.
4. Run it: `npm start`
5. In a **new terminal**, verify: `curl -v http://localhost:3000/health`

Stage 2: Externalize configuration

Goal: externalize the configuration by loading `PORT` and `ENVIRONMENT` from a `.env` file instead.

1. `npm i dotenv`
2. `touch .env`:

```
PORT=5000
```

```
ENVIRONMENT=development
```

3. Update your config so that:
   - `PORT` falls back to `3000` if not set externally.
   - `ENVIRONMENT` falls back to `'default'` if not set externally.
4. Add `environment` to the `/health` response body.
5. Run the app again and `curl -v http://localhost:5000/health`.

Reflection

- How would you easily be able to tell if the config was loaded correctly, just by looking at the response?
- What's the benefit of providing config externally rather than hardcoding it into the project?
- Why do we prefer to make our endpoints `async`?

Stage 3: Verification through integration (E2E) testing

Goal: restructure the project and add verification with Supertest.

Target structure

```
project-root/
├── src/
│   ├── app.js        # Express app, exports app (no .listen())
│   └── server.js      # Requires app.js, calls app.listen()
├── tests/
│   └── health.test.js
├── .env
├── package.json
```

Steps

1. `npm i --save-dev jest supertest`
2. Create a `src/` folder. Move your app into `src/app.js`, remove `app.listen(...)` from it, and export the app (`module.exports = app`). Create `src/server.js`, require the app from `./app`, and call `app.listen(...)` there instead.
3. Update your `start` script in `package.json` to point at `src/server.js`. Run it manually to confirm it still works before moving on.
4. Add a `test` script in `package.json` that runs `jest`.
5. Create `tests/health.test.js`:
   - Import the app: `const app = require('../src/app')`
   - Write a test that checks the response has status `200` and a body with `status: 'ok'`.
   - Write a second test that checks the response body has an `environment` property, **and** that its value is `.not.toBe('default')`.

Test structure to follow:

```
describe('GET /health', () => {
  it('first test description', async () => {
    const res = await ...
    ...
  });

  it('second test description', async () => {
    ...
  });
});
```

6. Run `npm test` and confirm both tests pass.

Reflection

- Why do we create separate `app.js` and `server.js` files? Think of at least two reasons (hint: separation of concerns).
- What benefit do we get from creating the `src` and `tests` folders? How would a project scale badly (more features, more files) without this structure?
- There are two places in this project where we read external config (`process.env`) — `src/app.js` and `src/server.js`. Do we need to load `dotenv` in both? What does this tell you about how the `dotenv` package actually works?
- Why assert the `environment` value is **not** `'default'`, instead of just checking the property exists? What would that weaker check miss?
- Try renaming the `tests` folder to something else, then run your tests. Now rename `health.test.js` to `health.e2e.js` instead. What happens in each case, and what does that tell you about how Jest finds tests?
- If you were to commit this project to Git, which files or folders would you want to `.gitignore`? Why those specifically?
- Why do we install jest and supertest with --save-dev instead of a regular npm i? What's different about where each type of dependency is needed?